

Concept of Abstract Data Types (ADT)

Q. Explain the concept of Abstract Data Type (ADT) with suitable examples. Why is it important in data structures?

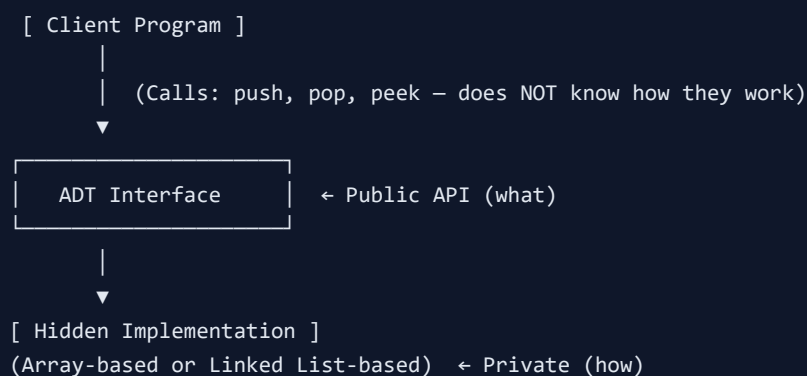
✦ Definition to Remember

An **Abstract Data Type (ADT)** is a conceptual model that defines a data type by specifying **what data is stored** and **what operations can be performed**, without specifying **how** these operations are implemented. The implementation is hidden from the user — only the interface is exposed.

1. Key Concepts of ADT

- **Abstract** = Implementation details are hidden from the user.
- An ADT consists of two parts:
- **Declaration of Data:** The structure and type of the data values stored.
- **Declaration of Operations:** The functions/methods that manipulate the data.
- **Encapsulation:** Data and operations are bundled together.
- **Data Hiding:** The internal working is completely hidden.
- **Separation of Concerns:** Divides the program into interface (client) and implementation (developer).

2. How ADT Works



3. Classic Example — Stack ADT

Component	Detail
Data	Collection of elements of the same type
push(item)	Inserts an item at the top
pop()	Removes and returns the top item
peek()	Returns top item without removing
isEmpty()	Checks if the stack is empty

The user calls `push()` and `pop()` without knowing if the stack is internally implemented using an **array** or a **linked list** — that detail is abstracted away.

4. Advantages of ADT

- **Code Reusability:** ADT implementation can be reused across different programs.
- **Modularity:** Changes to internal implementation don't affect the client program.
- **Simplicity:** Complex systems become easier to manage by hiding low-level details.

★ Must-Write Points (for 10 marks)

1. ADT defines *what* operations can be done, not *how* — implementation is hidden.
2. Two parts: **Declaration of Data** and **Declaration of Operations**.
3. ADT provides **Encapsulation** (data + operations together) and **Data Hiding**.
4. Classic examples: Stack ADT (push, pop, peek, isEmpty), Queue ADT (enqueue, dequeue).
5. Users interact only with the interface — they don't need to know the internal structure.
6. **Modularity:** changing implementation (array → linked list) doesn't break client code.
7. ADT promotes code reusability and simplifies complex systems.

⚡ Quick Recall

ADT → Defines What (not How) → Data + Operations → Encapsulation + Data Hiding → Stack ADT
(push/pop/peek) → Reusability + Modularity + Simplicity